

Prophet: Prompting Large Language Models With Complementary Answer Heuristics for Knowledge-Based Visual Question Answering

Zhou Yu¹, Member, IEEE, Xuecheng Ouyang, Zhenwei Shao¹,
Meng Wang², Fellow, IEEE, and Jun Yu³, Senior Member, IEEE

Abstract—Knowledge-based visual question answering (VQA) requires external knowledge beyond the image to answer the question. Early studies retrieve required knowledge from explicit knowledge bases (KBs), which often introduces irrelevant information to the question, hence restricting the performance of their models. Recent works have resorted to using a powerful large language model (LLM) as an implicit knowledge engine to acquire the necessary knowledge for answering. Despite the encouraging results achieved by these methods, we argue that they have not fully activated the capacity of the LLM as the provided textual input is insufficient to depict the required visual information to answer the question. In this paper, we present Prophet—a conceptually simple, flexible, and general framework designed to prompt LLM with answer heuristics for knowledge-based VQA. Specifically, we first train a vanilla VQA model on a specific knowledge-based VQA dataset without external knowledge. After that, we extract two types of complementary answer heuristics from the VQA model: answer candidates and answer-aware examples. The two types of answer heuristics are jointly encoded into a formatted prompt to facilitate the LLM’s understanding of both the image and question, thus generating a more accurate answer. By incorporating the state-of-the-art LLM GPT-3 (Brown et al. 2020), Prophet significantly outperforms existing state-of-the-art methods on four challenging knowledge-based VQA datasets. Prophet is general that can be instantiated with the combinations of different VQA models (i.e., both discriminative and generative ones) and different LLMs (i.e., both commercial and open-source ones). Moreover, Prophet can also be integrated with modern large multimodal models in different stages, which is named Prophet++, to further improve the capabilities on knowledge-based VQA tasks.

Index Terms—Visual question answering (VQA), knowledge-based VQA, large language models (LLMs), large multimodal models.

I. INTRODUCTION

RECENT advances in multimodal learning have achieved remarkable progress in various vision-language tasks, including visual captioning [2], [3], visual grounding [4], [5], [6], and visual question answering (VQA) [7], [8]. Among these tasks, VQA poses unique challenges by requiring machines to answer free-form questions through reasoning about given images. Benefiting from large-scale vision-language pretraining [6], [9], [10], the state-of-the-art methods have even surpassed human level on several representative benchmarks [11], [12]. Despite the success of these methods, their reasoning abilities are far from satisfactory, especially when *external knowledge* is required to answer the questions. In this situation, the task of knowledge-based VQA is introduced to validate models’ abilities to leverage external knowledge. Early knowledge-based VQA benchmarks additionally provide structured knowledge bases (KBs) and annotate required knowledge facts for all the questions [13], [14]. More recently, benchmarks emphasizing on *open-domain* knowledge have been established [15], [16], which means KBs are no longer provided and any external knowledge resource can be used for answering. We focus on the task with open-domain knowledge in this paper.

A straightforward solution for knowledge-based VQA is to retrieve knowledge entries from explicit KBs, e.g., Wikipedia and ConceptNet [17]. Then, a KB-augmented VQA model performs joint reasoning over the retrieved knowledge, image, and question to predict the answer [18], [19], [20]. However, the performance of these retrieval-based approaches is limited for two reasons: (i) the required knowledge may not be successfully retrieved from the KBs; and (ii) even if the required knowledge is retrieved, plenty of irrelevant knowledge is inevitably introduced, which hampers the learning of VQA models.

Apart from those studies using explicit KBs, another line of research resorts to pretrained large language models (LLMs), e.g., GPT-3 [1], as implicit knowledge engines for knowledge acquisition. A pioneering work by PICa employs the frozen GPT-3 model to answer the question with a formatted prompt

Received 6 March 2024; revised 22 March 2025; accepted 15 April 2025. Date of publication 18 April 2025; date of current version 3 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62422204, Grant 62125201, Grant U24B20174, Grant 62072147, Grant 62020106007, and in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LR22F020001 and Grant LDT23F02025F02. Recommended for acceptance by R. Cucchiara. (Corresponding author: Jun Yu.)

Zhou Yu and Zhenwei Shao are with the Key Laboratory of Complex Systems Modeling and Simulation, School of Computer Science, Hangzhou Dianzi University, Zhejiang 310005, China (e-mail: yuz@hdu.edu.cn; shaozw@hdu.edu.cn).

Xuecheng Ouyang is with HDU-ITMO Joint Institute, Hangzhou Dianzi University, Zhejiang 310005, China (e-mail: ouyangxc@hdu.edu.cn).

Meng Wang is with the School of Computer Science and Information Engineering, Hefei University of Technology, Anhui 230002, China (e-mail: eric.mengwang@gmail.com).

Jun Yu is with the School of Intelligence Science and Engineering, Harbin Institute of Technology, Shenzhen 518055, China (e-mail: yujun@hit.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2025.3562422>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2025.3562422

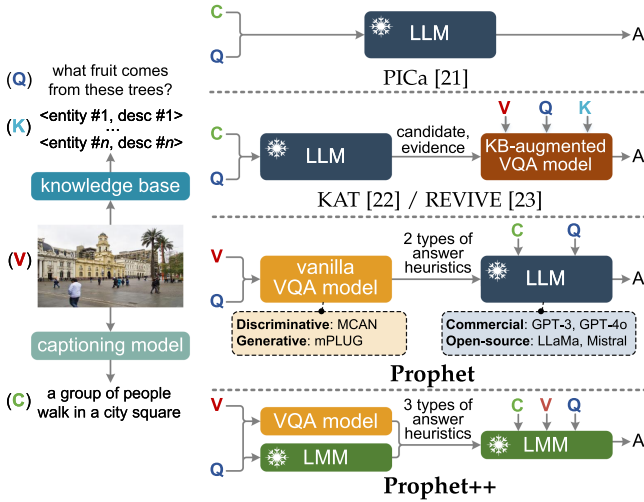


Fig. 1. Conceptual comparisons of three knowledge-based VQA frameworks using a frozen LLM model, e.g., GPT-3 [1]. While PICa [21], KAT [22], and REVIVE [23] directly feed the caption (C) and question (Q) into the LLM as the prompt, we argue that the information they provide for the LLM is insufficient thus cannot fully activate the LLM’s potential. In contrast, our Prophet learns a vanilla VQA model without external knowledge to produce *answer heuristics*, which endows the LLM with richer and more task-specific information for answer prediction. In contrast to the counterparts that resort to specific VQA models and LLMs, Prophet is general that can be instantiated with the combinations of different VQA models (i.e., both discriminative and generative ones) and different LLMs (i.e., both commercial and open-source ones). Moreover, Prophet can also be integrated with large multimodal models (LMMs) in different stages, which is termed Prophet++, to further improve the capabilities on knowledge-based VQA tasks.

as its input [21]. Given a testing image-question pair, PICa first translates the image into a caption using an off-the-shelf captioning model. The question, caption, and a few in-context examples are then integrated into a textual prompt that can induce GPT-3 to predict the answer directly. Thanks to the powerful knowledge reasoning ability of GPT-3, PICa achieves significant performance improvements compared to those retrieval-based methods using explicit KBs. Inspired by PICa, KAT [22] and REVIVE [23] learn KB-augmented VQA models to exploit both the implicit knowledge from LLMs and explicit knowledge from KBs for answer prediction. The synergy of the two knowledge resources brings further improvements to their models. Despite the promising results achieved by these methods, they have not fully activated the capability of the LLMs due to the following limitations:

- 1) The captions cannot cover all the necessary information in the image. Consider the example in Fig. 1, the caption “a group of people walk in a city square” contributes nothing to answering the question “what fruit comes from these trees”. In such situation, the LLM has to make an aimless and biased guess to answer the question.
- 2) LLMs like GPT-3 employ a few-shot learning paradigm that requires a few in-context examples to adapt to new tasks. Therefore, the choice of these examples is critical to model performance. As reported in [21], existing example selection strategies achieve far inferior performance to the oracle strategy that selects examples for a testing sample

based on the similarity of their ground-truth answers, which is unavailable during testing.

We ask: *Is it possible to endow the LLM with some heuristics to enhance its capacity for knowledge-based VQA?*

In this paper, we present **Prophet**—a conceptually simple yet effective framework designed to **prompt** LLMs with answer **heuristics** for knowledge-based VQA. By answer heuristics, we mean some promising answers that are presented in a proper manner in the prompt. Specifically, we introduce two types of complementary answer heuristics, namely *answer candidates* and *answer-aware examples*, to overcome the limitations in (i) and (ii), respectively. Given a testing input consisting of an image and a question, the answer candidates refer to a list of promising answers to the testing input, where each answer is associated with a confidence score. The answer-aware examples refer to a list of in-context examples, where each example has a similar answer to the testing input. Fortunately, these two types of answer heuristics can be simultaneously obtained from any vanilla VQA model trained on a specific knowledge-based VQA dataset. A schematic of Prophet is illustrated in Fig. 1.

Without bells and whistles, Prophet surpasses previous state-of-the-art single-model results on the challenging OK-VQA and A-OKVQA datasets [15], [16], including the heavily-engineered Flamingo-80B model trained on 1.8B image-text pairs [11]. Moreover, Prophet is friendly to most researchers, as our results can be reproduced using a single GPU and a number of GPT-3 invocations.

A preliminary version of this manuscript was published in [24]. Based on that version, we have made the following contributions to further improve the capabilities and generality of Prophet: (i) we investigate diverse types of VQA models, including the classical discriminative models trained from scratch and the latest generative VQA models pretrained on large-scale corpus; (ii) we expand the used LLM from the commercial GPT-3 model to a wide range of open-source models; (iii) apart from OK-VQA and A-OKVQA, we conduct more experiments on two other knowledge-based VQA datasets, namely ScienceQA [25] and TextVQA [26]. Furthermore, we present an improved Prophet++ framework by introducing large multimodal models (LMMs) into Prophet, which generates a new type of answer heuristic (i.e., *answer-aware rationales*) in the first stage and enables an extra visual input in the second stage.

The source code is made available here.¹ We hope our studies may inspire future research on knowledge-based VQA and universal vision-language learning.

II. RELATED WORK

Visual Question Answering (VQA): VQA has been of growing interest over the last few years. Recent studies in VQA research can be roughly divided into the following categories: better visual features [2], [27], [28], more powerful model architectures [7], [8], [29], and more effective learning paradigms [9], [30], [31], [32], [33]. Most current state-of-the-art VQA methods employ the Transformer architecture [34]. By incorporating

¹<https://github.com/MILVLG/prophet>

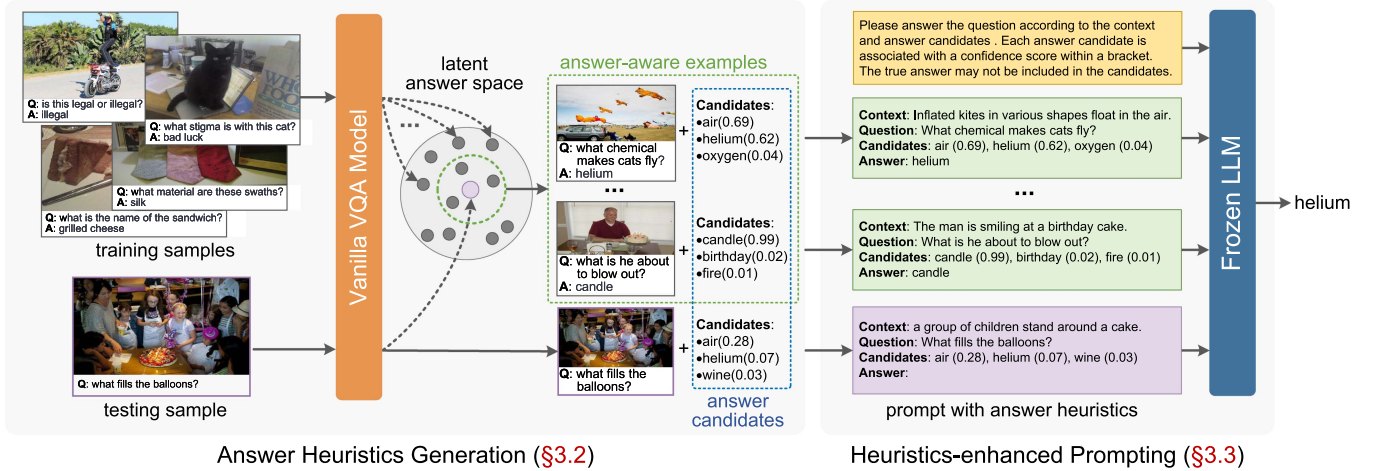


Fig. 2. Our Prophet framework has two stages: answer heuristics generation and heuristics-enhanced prompting. In the answer heuristics generation stage, a vanilla VQA model trained on specific knowledge-based VQA dataset is employed to generate two types of complementary answer heuristics, i.e., answer candidates and answer-aware examples. In the heuristics-enhanced prompting stage, the answer heuristics, question, and caption are integrated into a formatted prompt to instruct a frozen LLM (e.g., GPT-3) to predict an answer. As shown in the example, both answer heuristics contribute to the answer of “helium”.

vision-language pretraining on large-scale datasets, they have approached or even surpassed human-level performance on several representative benchmarks [11], [12]. Besides these studies on general-purpose VQA, there is also a growing trend towards exploring more granular VQA tasks with specific reasoning skills, e.g., neural-symbolic reasoning [35], [36] and knowledge utilization [13], [15].

Knowledge-based VQA: The core of this task lies in knowledge acquisition and integration. Early explorations parse the inputs into structured queries and retrieve supporting knowledge from fixed knowledge bases (KBs) to obtain the answers [13], [14]. As the provided knowledge resources are not sufficient to represent general knowledge, subsequent research mainly focuses on acquiring explicit knowledge from multiple open-domain knowledge resources, e.g., ConceptNet [17], Wikipedia [37], and Google Images [19]. This retrieved knowledge is integrated with the image-question pair for answer prediction [19], [20]. Motivated by the powerful capacities of LLMs (e.g., GPT-3 [1]) in knowledge reasoning, recent state-of-the-art approaches regard an LLM as an implicit knowledge engine. They either utilize it to predict answers from given questions and extract visual captions [21] or to extract answer candidates with evidence to improve answer prediction [22], [23]. Nevertheless, they have not fully activated the reasoning capability of LLMs, as the necessary visual information to answer the question is not represented exactly.

This motivates us to explore the strategies for prompting LLMs with question-aware information (i.e., answer heuristics). Similar to Prophet, a concurrent work PromptCap also aims to enhance the input information for LLMs by learning a question-aware captioning model [38]. However, PromptCap needs to use LLM in both the training and testing phases, which incurs tremendous computational costs as the training set is usually large. In contrast, Prophet is more economical as it only utilizes LLM in the testing phase.

In-context learning: Unlike the *pretrain-then-finetune* paradigm for language models like BERT [39], GPT-3 innovatively introduces a *few-shot in-context learning* paradigm and has become the de facto standard for subsequent LLMs. To adapt to a new task, GPT-3 only needs to concatenate a few examples of the task with the input as the *prompt* at inference time and requires no parameter updates. This appealing property has inspired research on training multimodal few-shot learners [11]. Empirical studies show that a huge model (e.g., 80B parameters in Flamingo [11]) is required for effective few-shot learning, which is unaffordable for most people to reproduce their results.

III. THE PROPHET AND PROPHET++ FRAMEWORKS

As depicted in Fig. 2, our Prophet is a conceptually simple two-stage framework. In the answer heuristics generation stage, a vanilla VQA model is learned to generate two types of answer heuristics, i.e., answer candidates and answer-aware examples (detailed in Section III-B). In the heuristics-enhanced prompting stage, the answer heuristics, question, and caption are integrated into a formatted prompt to instruct a frozen LLM to predict an answer (detailed in Section III-C). Moreover, we introduce modern large multimodal models (LMMs) in different stages of Prophet to obtain a more powerful Prophet++ framework, which is detailed in Section III-D.

A. Preliminaries

Before presenting the Prophet, we briefly introduce the in-context learning paradigm developed by GPT-3 and its adaptation to knowledge-based VQA by PICa [21].

GPT-3 is an autoregressive language model pretrained on a tremendous dataset. During inference, in-context few-shot learning formulates a new downstream task as a text sequence generation task on the frozen model. Given a testing input x , its target y is predicted conditioned on a formatted prompt $p(h, \mathcal{E}, x)$, where h refers to a prompt head, *aka* instruction, that

describes the task, $\mathcal{E} = \{e_1, \dots, e_n\}$ corresponds to n in-context examples. Let the target $\mathbf{y} = (y^1, \dots, y^L)$ be a text sequence of L tokens. For notational convenience, we denote $[l]$ as a set of natural numbers from 1 to l and use $\mathbf{y}^{[l]} = (y^1, \dots, y^l)$ to represent a sub-sequence containing the first l words of $\mathbf{y}$. At each decoding step l , we have:

$$y^l = \underset{\hat{y}^l}{\operatorname{argmax}} p_{\text{GPT-3}}(\hat{y}^l | \mathbf{p}, \mathbf{y}^{[l-1]}) \quad (1)$$

where each in-context example $e_i = (x_i, y_i)$ contains an input-target pair of the task, which is constructed manually or sampled from the training set.

To adapt LLMs like GPT-3 to address the knowledge-based VQA task, the key is to design proper prompts. Given a question q and an image v as inputs, the VQA task aims to predict a target answer a . Since LLMs do not understand images intrinsically, the image needs to be translated into a caption c using an off-the-shelf captioning model. PICa formulates the testing input x as the following template:

Context: c \n Question: q \n Answer:

where the variables marked in blue will be substituted by specific testing inputs. \n stands for a line break in the template. Accordingly, each in-context example e_i is formulated into a similar template as follows:

Context: c_i \n Question: q_i \n Answer: a_i

where c_i , q_i , and a_i refer to an image-question-answer triplet collected from the training set. The complete prompt of PICa consists of a fixed prompt head, a few in-context examples, and a testing input. This prompt is fed into a frozen LLM for answer prediction.

Our Prophet inherits the pipeline of PICa. In addition, we introduce answer heuristics into the prompt structure to better activate the reasoning capability of the LLM, which leads to more accurate answers.

B. Stage-I: Answer Heuristics Generation

We introduce two types of answer heuristics: answer candidates and answer-aware examples. Given a testing input consisting of an image and a question, the answer candidates refer to a list of promising answers to the testing input, where each answer is associated with a confidence score. The answer-aware examples refer to a list of in-context examples, where each example has similar answers to the testing input. Interestingly, these two types of answer heuristics can be obtained simultaneously from any vanilla VQA model trained on specific knowledge-based VQA task.

As shown in Fig. 3, existing VQA methods can be categorized into *discriminative* and *generative* ones based on the ways they obtain answers. This discrepancy leads to different strategies for answer heuristics generation. We elaborate the strategy for each class of VQA models below.

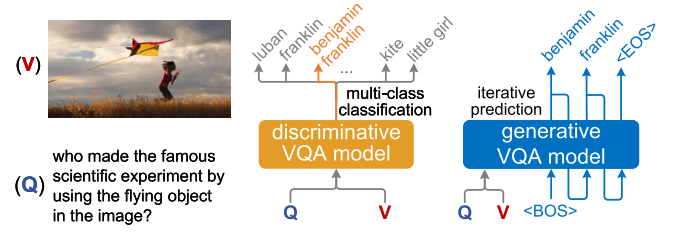


Fig. 3. Discriminative vs. generative VQA models. Taking an image (V) and a question (Q) as inputs, a typical discriminative VQA model performs multi-class classification to predict the most relevant answer (may contain multiple words) from a predefined answer vocabulary, while a typical generative VQA model iteratively predicts one answer word at a time to constitute the final answer.

1) *Discriminative VQA Models*: Denote a VQA training dataset as $\mathcal{D} = \{(v_i, q_i, a_i)\}_{i=1}^M$, where v_i, q_i, a_i refer to the image, question, and answer, respectively. The most frequent answers in the training set form an answer vocabulary $\mathcal{V} = \{w_j\}_{j=1}^S$, where S is the answer vocabulary size. A discriminative VQA model $\mathcal{M}_{\text{disc}}$ is learned from $\mathcal{D}$ to perform an S -way classification over the answers. Generally, the model $\mathcal{M}_{\text{disc}}$ can be separated into two submodels, i.e., a backbone $\mathcal{M}_{\text{disc}}^B$ and a prediction head $\mathcal{M}_{\text{disc}}^H$. The backbone $\mathcal{M}_{\text{disc}}^B$ acts as an encoder to fuse multimodal inputs v and q and obtain a fused feature z :

$$z = \mathcal{M}_{\text{disc}}^B(v, q) \quad (2)$$

The prediction head $\mathcal{M}_H$ simply adopts a linear layer followed by a sigmoid function to project the fused feature z into a score vector $y \in \mathbb{R}^S$ over the answer vocabulary:

$$y = \mathcal{M}_{\text{disc}}^H(z) \quad (3)$$

where the i th element of y represents the confidence score for answer w_i . Based on the above definitions, we explain how to generate the two types of answer heuristics below. Note that although the learned VQA model $\mathcal{M}_{\text{disc}}$ does not incorporate any external knowledge, it can be used for knowledge-based VQA when trained properly. We regard it as a reference model and compare its performance to Prophet in the experiments to show the effectiveness of LLM for knowledge-based VQA.

Answer candidates: Given a testing input (v, q) , we obtain its score vector y for all answers using (3). Denoting $s_i \in \mathbb{R}^+$ as the i th element of y , we obtain the top- K answers with the highest scores as follows:

$$\mathcal{I}_{\text{AC}} = \underset{j \in \{1, 2, \dots, S\}}{\operatorname{argTopK}} s_j \quad (4)$$

where $\mathcal{I}_{\text{AC}}$ denotes an index set of the top- K answer candidates. The answer candidates $\mathcal{C}$ are defined as follows:

$$\mathcal{C} = \{(w_j, s_j) \mid j \in \mathcal{I}_{\text{AC}}\} \quad (5)$$

where w_j and s_j are an answer candidate and its confidence score, respectively. To make the formats of the in-context examples and testing input consistent, for each example e_i we also calculate and provide a set of answer candidates $\mathcal{C}_i$.

Answer-aware examples: Several previous studies have shown that the choice of in-context examples is crucial for LLM's few-shot learning [21]. Their results motivate us to devise an *answer-aware* example selection strategy.

Given a testing input (v, q) and any training input (v_i, q_i) , we can obtain their corresponding fused features z and z_i from (2) using the trained model. Since the fused features are linearly projected for answer prediction, we conjecture that these fused features lie in a *latent answer space* that contains rich semantics of the answers to the given image-question pairs. If z and z_i are close in the latent space, they are more likely to share similar answers and image-question inputs.

We calculate the cosine similarity of the fused feature between the testing input and each training input, then select top- N nearest neighbors in the latent space as the answer-aware examples:

$$\mathcal{I}_{\text{AE}} = \underset{i \in \{1, 2, \dots, M\}}{\text{argTopN}} \frac{z^T z_i}{\|z\|_2 \|z_i\|_2} \quad (6)$$

where $\mathcal{I}_{\text{AE}}$ is an index set of the top- N similar samples in $\mathcal{D}$. The answer-aware examples $\mathcal{E}$ are defined as follows:

$$\mathcal{E} = \{(v_i, q_i, a_i) \mid i \in \mathcal{I}_{\text{AE}}\} \quad (7)$$

Note that the fused features of the training inputs can be computed and stored beforehand, allowing efficient answer-aware example selection.

2) *Generative VQA Models:* Recent state-of-the-art VQA models tend to use generative model architectures due to their remarkable scalability and generalizability [12], [30], [40].

Given the same VQA training dataset $\mathcal{D} = \{(v_i, q_i, a_i)\}_{i=1}^M$ as above, a generative VQA model $\mathcal{M}_{\text{gen}}$ is learned from $\mathcal{D}$ to generate answers word-by-word from a pre-defined word vocabulary $\mathcal{V} = \{w_j\}_{j=1}^S$, where S is the word vocabulary size. Each answer can be represented as a text sequence with a dynamic length of L words:

$$\mathbf{w} = (w^1, w^2, \dots, w^L) \quad (8)$$

where $w^1 = [\text{BOS}]$ refers to a special start-of-sentence token and $w^L = [\text{EOS}]$ refers to an end-of-sentence token.

Similar to the discriminative model, $\mathcal{M}_{\text{gen}}$ can also be separated into a backbone $\mathcal{M}_{\text{gen}}^B$ and a prediction head $\mathcal{M}_{\text{gen}}^H$. The backbone $\mathcal{M}_{\text{gen}}^B$ corresponds to an encoder-decoder or a pure decoder architecture that fuses multimodal inputs v and q , and then generates latent feature of each answer word using an autoregressive manner:

$$z^l = \mathcal{M}_{\text{gen}}^B(v, q, \mathbf{w}^{[l-1]}) \quad (9)$$

where z^l denotes the latent feature of l th answer word. On top of the latent feature z^l , the prediction head $\mathcal{M}_{\text{gen}}^H$ applies a linear projection (or a MLP) followed by a softmax function to decode it into a score distribution $y^l \in \mathbb{R}^S$ over the whole word vocabulary:

$$y^l = \mathcal{M}_{\text{gen}}^H(z^l) \quad (10)$$

where the l th answer word w^l is obtained from y^l by greedily choosing the word with the highest score. Until an [EOS] token

is generated, w^l is appended to $\mathbf{w}^{[l-1]}$ to obtain $\mathbf{w}^{[l]}$, which is iteratively fed into the model $\mathcal{M}_{\text{gen}}$ to predict the next word.

Answer candidates: Given a testing input (v, q) , we can obtain its most relevant answer using the greedy decoding strategy above. However, how to obtain the answer candidates consisting of the top- K answers and their confidence scores is not straightforward. We resort to the *beam search* algorithm, which is widely used in neural machine translation [41] and visual captioning [3], to address the issue.

Similar to (5), we denote the top- K answer candidates as a set of tuples as follows:

$$\mathcal{C} = \{(\mathbf{w}_1, s_1), (\mathbf{w}_2, s_2), \dots, (\mathbf{w}_K, s_K)\} \quad (11)$$

where each $\mathbf{w}_j$ represents an answer consisting of a sequence of answer words and $s_j \in \mathbb{R}^+$ denotes its corresponding confidence score calculated over all the answer words. The answer candidate set $\mathcal{C}$ is obtained from the generative model $\mathcal{M}_{\text{gen}}$ equipped with the beam search strategy.

Specifically, we initialize each answer $\mathbf{w}_j$ with the same [BOS] token. At each decoding step l , each $\mathbf{w}_j$ of length l is first passed through $\mathcal{M}_{\text{gen}}$ to obtain its top- K candidate words with the highest scores. After that, an *expand-then-reduce* strategy is performed to update the K answers: (i) **expand step:** each $\mathbf{w}_j$ is expanded K times to combine with the K candidate words, resulting in $K * K$ new candidate answers of length $l + 1$; (ii) **reduce step:** among the $K * K$ candidate answers, only the top- K ones with the highest accumulated scores $s = \sum_{i=1}^l \log y_i$ are retained, which are then regarded as the inputs to the next decoding step.

Answer-aware examples: Similar to the example selection strategy for discriminative models, the answer-aware examples for generative models are also obtained by performing kNN search in a latent answer space. It is worth noting that the granularity of the latent features is different for the two types of VQA models: each latent feature obtained from a discriminative VQA model refers to an answer entry in the answer vocabulary, while each latent feature obtained from a generative VQA model refers to an answer word.

Given a testing input (v, q) and i th training input (v_i, q_i) , the latent features for their multi-word answers can be respectively represented as feature groups $Z = [z^1, z^2, \dots, z^L] \in \mathbb{R}^{L \times d}$ and $Z_i = [z_i^1, z_i^2, \dots, z_i^{L_i}] \in \mathbb{R}^{L_i \times d}$, where d is the common dimensionality of the latent answer space, L and L_i refer to the answer lengths of Z and Z_i , respectively. We define a simple score function as follows to average the dot-product similarity of each paired features $z_j \in Z$ and $z_i^k \in Z_i$:

$$\pi_i = \frac{1}{L * L_i} \sum_{j=1}^L \sum_{k=1}^{L_i} \frac{z_j^T z_i^k}{\|z_j\|_2 \|z_i^k\|_2} \quad (12)$$

Using (12), we obtain the top- N nearest neighbors of the query input in the training set and then format them as the answer-aware examples $\mathcal{E}$ as follows:

$$\begin{aligned} \mathcal{I}_{\text{AE}} &= \underset{i \in \{1, 2, \dots, M\}}{\text{argTopN}} \pi_i \\ \mathcal{E} &= \{(v_i, q_i, a_i) \mid i \in \mathcal{I}_{\text{AE}}\} \end{aligned} \quad (13)$$

where $\mathcal{I}_{AE}$ is an index set of the top- N nearest neighbors in the training set $\mathcal{D}$.

C. Stage-2: Heuristics-Enhanced Prompting

After obtaining the answer heuristics (i.e., answer candidates $\mathcal{C}$ and answer-aware examples $\mathcal{E}$) from the stage-1, we encode them into a heuristics-enhanced prompt to facilitate the few-shot learning capacity of the LLM for knowledge-based VQA.

A prompt consists of a prompt head, a set of in-context examples, and a testing input. The prompt head describes the VQA task in natural language. We refer to the prompt head designed in PICa and supplement it with a new description of the answer candidates. Although we encourage LLM to generate answers according to the answer candidates, we also allow it to take broad explorations and generate answers beyond the candidates. The complete format of our prompt head is shown in the yellow box of Fig. 2.

Our in-context examples are derived from the obtained N answer-aware examples $\mathcal{E} = \{e_1, e_2, \dots, e_N\}$. Based on PICa's template in Section III-A, for example e_i , we introduce its answer candidates $\mathcal{C}_i$ by adding *one* line of code as follows:

Context: c_i \n Question: q_i \n
 Candidates: $w_{j_1}(s_{j_1}), w_{j_2}(s_{j_2}), \dots, w_{j_K}(s_{j_K})$ \n
 Answer: a_i

where $j_1, j_2, \dots, j_K$ correspond to the actual indices of the elements in $\mathcal{C}_i$. Each answer candidate w_{j_k} is paired with its confidence score s_{j_k} within a bracket. The confidence scores additionally offer the reliability of the corresponding answer candidates, which helps the LLM focus more on the promising candidates and be more tolerant of the less relevant candidates. For the testing input, its template is similar to that for the in-context examples, except that the answer slot is left blank for the LLM to fill with.

To better exploit available examples, we use the multi-query ensemble strategy [21]. Specifically, we increase the number of answer-aware examples to $N \times T$ to obtain T paralleled prompts, where each prompt still contains N examples. By prompting the LLM for T times, we obtain T answer predictions. The majority voting is performed over the T predictions to determine the final answer. The effects of different N and T will be verified in the experiments.

D. Prophet++ With Frozen Large Multimodal Models

The original design of Prophet is based on pure-text LLMs that cannot perceive the images. Recently, a series of large multimodal models (LMMs), e.g., GPT-4 V, GPT-4o, and LLaVA [42], have been proposed and show remarkable capabilities on various multimodal tasks. This raises a natural question: “Can these powerful LMMs be utilized in Prophet to further facilitate its capabilities?” To this end, we propose Prophet++, an extended framework that utilizes LMMs in both stages of Prophet, as shown in Fig. 4.

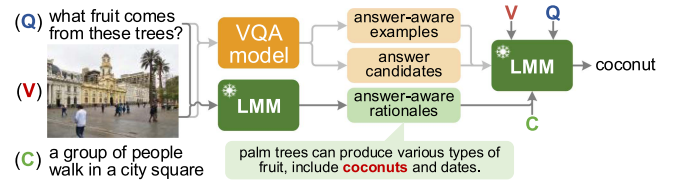


Fig. 4. The Prophet++ framework additionally introduces large multimodal models (LMMs) in different stages of Prophet. Specifically, the LLM in stage-1 is used to generate a new type of answer heuristic, i.e., the *answer-aware rationales* while the LLM in stage-2 is used to handle an extra visual input (V), thus providing more comprehensive knowledge to answer the question.

In the first stage of Prophet++, we first use the VQA model to extract the two aforementioned answer heuristics in Prophet. Inspired by the strong reasoning capability of the latest LMMs, we extract a new type of answer heuristic, i.e., the *answer-aware rationales*, which contains the reasoning process to answer the question. The prompt to generate answer-aware rationales is as follows:

You are provided with an image and a question. Please think step by step to generate the rationale to help answering the question.

where a few examples are also provided to control the output format. The output rationale is appended to the extracted image caption to be used in the second stage.

In the second stage of Prophet++, we replace the frozen LLM in Prophet with an LMM to handle multimodal inputs. Note that it takes a considerable amount of tokens to encode an image by a typical LLM. To restrict the total length of the multimodal inputs, only the test image is fed into the LLM while the images of the in-context examples are omitted.

IV. EXPERIMENTS

We mainly evaluate the performance of Prophet on two prevalent knowledge-based VQA datasets: OK-VQA [15] and A-OKVQA [16]. We conduct comprehensive ablation experiments to explore the effectiveness of Prophet. By taking the ablation results into account, we perform thorough comparisons of Prophet and state-of-the-art methods. Moreover, we showcase the generalization ability of Prophet on two diverse knowledge-based VQA datasets ScienceQA [25] and Text-VQA [26], which require external science and OCR knowledge, respectively.

A. Datasets

OK-VQA is a commonly used knowledge-based VQA dataset [15]. The dataset contains 9 K and 5 K image-question pairs for training and testing, respectively. All questions are manually filtered to ensure that outside knowledge is required to answer the questions.

A-OKVQA is currently the largest knowledge-based VQA dataset [16]. The dataset is split into three subsets: 17 K training,

1 K validation, and 7 K testing. Both direct answering (DA) and multiple choice (MC) evaluations are provided.

ScienceQA is a dataset that consists of about 21 K questions over a diverse set of high school-level science topics [25]. Out of the 21K questions, only the ‘IMG’ subset of 10.3K (48.7%) samples with images are used in our experiments.

TextVQA contains 28K images and 45K questions, where each question requires models to read and reason about the text in the image [26]. The dataset is split into three subsets of 34.6K training, 5K validation, and 5.7K testing questions. Following [43], [44], we supplement the training set with the augmented VQA samples from ST-VQA [45].

B. Implementation Details

Default settings on OK-VQA: We use the MCAN-large [8] as our default VQA model to generate answer heuristics. To improve the model capability, we modify the original MCAN model by: (i) replacing the original bottom-up-attention region-based features with the grid-based features extracted from CLIP’s visual encoder with a RN50×64 backbone [46]; and (ii) replacing the original LSTM network with a pretrained BERT-large model [39].

Similar to [18], we apply the transfer learning paradigm to further enhance the model capability. The model is first pretrained on the VQAv2 dataset [47] and Visual Genome dataset [48]. To prevent data contamination, we remove those samples from the pretraining dataset, whose images are used in the testing split of OK-VQA. After that, the pretrained model is further finetuned on the training split of OK-VQA to obtain our final VQA model. Note that the answer vocabulary of the pretrained model (with 3,129 answers) is quite different from the vocabulary of OK-VQA. To bridge this gap, we merge the answer vocabulary of OK-VQA² with the existing vocabulary, resulting in an expanded answer vocabulary with 4,477 answers for model finetuning. This model is trained on a *single* Nvidia RTX 3090 GPU, which is affordable for most people.

During the prompting stage using LLMs, we follow PICa to use OSCAR+ as the captioning model [27]. Unless otherwise noted, we set the number of answer candidates $K=10$, the number of in-context examples $N=16$, and the number of queries $T=5$ as our default settings. The default version of GPT-3 used in our experiments is `text-davinci-002` and the sampling temperature is set to 0.

Settings on other datasets: The settings and strategies for OK-VQA can be directly transferred to A-OKVQA to address its DA task. For the MC task, we follow the strategy in [16] to project the predicted answer to the nearest answer choice. Moreover, we design a Prophet variant for the MC task. It uses a slightly different prompt by injecting the choices to in-context examples and testing input, and instructs the LLM to *choose* the correct one from four choices.

For ScienceQA, we reuse all the default settings for OK-VQA. If a training sample provides extra textual hint, we simply append

²Similar to [2], we collect answers that appear more than eight times in the training set of OK-VQA, resulting in 2,794 answers.

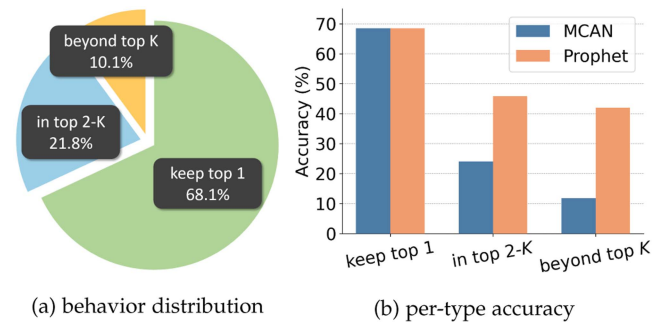


Fig. 5. Prophet’s prediction behaviors in terms of (a) distribution and (b) per-type accuracy. As Prophet takes K answer candidates from MCAN as inputs, we define three prediction behaviors of Prophet, namely “keep top 1”, “in top 2-K”, and “beyond top K” predictions of MCAN, respectively. Note that all the testing samples can be categorized into one of the three classes above.

the text to the generated caption as the new context of the corresponding image. For TextVQA, we use the commercial system from Amazon to extract OCR from images,³ whose effectiveness has been verified in previous work [44]. The extracted OCR texts are provided in both the in-context examples and testing input to instruct the LLM.

Settings for other Prophet and Prophet++ variants: In addition to MCAN, we also experiment on Prophet with a generative VQA model mPLUG [40], which is first pretrained on massive image-text pairs and then finetuned on specific VQA dataset. Following the aforementioned two-stage transfer learning paradigm for MCAN, the pretrained mPLUG model is first finetuned on the VQAv2 dataset and then further finetuned on specific knowledge-based VQA dataset. For Prophet++, we use the state-of-the-art LMM GPT-4o in both stages.

C. Ablation Studies

We conduct ablation experiments for Prophet and Prophet++ on OK-VQA using the default settings above if not mentioned otherwise. Results shown in Table I and Fig. 5 are discussed in detail below.

Prompting vs. retrieval: Prophet uses a prompting-based paradigm to predict the answer based on a set of promising answer candidates. In contrast, a previous work MAVEx [19] exploits answer candidates but adopts a retrieval-based paradigm to search knowledge from external KBs to determine the answer. As both Prophet and MAVEx train a VQA model to generate answer candidates (stage-1), we can compare the superiority of the two paradigms (stage-2). In Table I(a), we show the performance of the two paradigms in terms of stage-1 accuracy and final accuracy, respectively.

For a fair comparison, we re-implement the VQA model used in MAVEx, i.e., ViLBERT [31], to generate answer heuristics for our Prophet. From the results, we can see that based on the same VQA model, our Prophet outperforms MAVEx by a large margin (44.97% vs. 40.28%), showing the superiority of our prompting-based paradigm over MAVEx’s retrieval-based paradigm in external knowledge acquisition and integration.

³<https://aws.amazon.com/textextract/>

TABLE I
ABLATION EXPERIMENTS FOR PROPHET

VQA model, paradigm	stage-1 acc.	accuracy	visual features	stage-1 acc.	accuracy	#candidates (K)	hit rate	accuracy
ViLBERT, retrieval [19]	35.20	40.28 (+5.08)	Bottom-Up [2]	46.83	55.34 (+8.51)	0	-	49.63
ViLBERT, prompt [†]	35.28	44.97 (+9.69)	VinVL [27]	47.88	56.23 (+8.35)	1	53.04	56.04
			CLIP-ViT-L/14 [46]	52.03	60.12 (+8.09)	5	75.20	60.17
			CLIP-RN50×64 [46]	53.04	60.84 (+7.80)	10	79.83	60.84

example selection	hit rate	accuracy	#examples (N)	accuracy ($T=1$)	accuracy ($T=5$)	variants	accuracy
(a) rand	5.31	58.66	0	49.97	49.97	(a) default	60.84
(b) ques + img [21]	59.58	59.82	1	54.89	56.75	(b) w/o prompt head	60.54
(c) fused	83.63	60.84	8	57.49	59.91	(c) w/o confidence scores	55.46
(d) fused + ques + img	82.45	60.38	16	57.52	60.84	(d) w/o image captions	58.27
(e) answer logits	79.25	60.40	20	57.91	61.10	(e) default+tags [21]	60.51

(a) **Prompting vs. retrieval.** Our prompting-based paradigm is more effective than the retrieval-based one in MAVEx [19]. [†]: our re-implementation.

(b) **Capability of VQA models.** More powerful VQA models lead to higher accuracies, but obtain slightly less relative improvements from stage-2.

(c) **Answer candidates.** They are crucial to Prophet and increasing K leads to better performance.

(d) **Example selection strategy.** Our answer-aware example selection based on fused features is more effective than the others.

(e) **Numbers of examples and queries.** Increasing N and T improves model performance at the expense of linearly increasing overheads.

(f) **Prompt contents.** The default settings contain the exact necessary information for prompting.

All the reported results are evaluated on the testing set of OK-VQA v1.1. The best result in each table is bolded and the result with the default settings is marked in gray.

Capability of VQA models: In Table I(b), we study how the VQA models of different capabilities impact the performance of Prophet. To better control the model capability, we use the same MCAN model trained with four visual features: region-based Bottom-Up [2] and VinVL [27] features and grid-based CLIP features from two backbones (ViT-L/14 and RN50×64) [46]. Results show that more powerful VQA models (reflected in the stage-1 accuracies) lead to better performance of Prophet, as they provide answer heuristics of higher quality. Combining the results in Table I(a), we also observe that more powerful VQA models achieve less relative improvements from GPT-3, which can be explained by the intrinsic diminishing return property. As a by-product, we verify that the visual features are important to the performance of knowledge-based VQA, which is consistent with the observations in [23]. The models with CLIP-based visual features significantly outperform those with region-based features, indicating that the CLIP’s visual features contain richer visual knowledge due to large-scale pretraining.

In addition to using different visual features for MCAN, we can also replace the whole MCAN model with any generative models pretrained on large-scale multimodal datasets as mentioned in Section III-A. These results will be reported in the main results.

Answer candidates: Table I(c) varies the number of answer candidates K from 0 to 10 to explore its effect on Prophet. For each testing sample, if the ground-truth answer is hit by one of the K answer candidates, we accumulate the soft score of that ground-truth answer.⁴ The hit rate is calculated over the testing set by dividing the accumulated score by the number of samples.

From the results, we can see that: (i) without any answer candidates, Prophet’s accuracy drops by 6.4 points ($K=0$ vs. $K=1$), showing the importance of answer candidates in Prophet; (ii) with the increase of answer candidates, the hit rate and final accuracy grow accordingly but they exhibit a tendency

to saturate. This is because the quality of answer candidates eventually meets saturation as K increases; (iii) when $K=1$, the final accuracy is even higher than the hit rate (56.04% vs. 53.04%), which implies that GPT-3 has a strong capability to correct the wrong answer candidates while keeping the correct ones.

Example selection strategy: To show the effectiveness of our answer-aware example selection strategy, we compare it to other example selection strategies in Table I(d). The compared strategies include: (a) *rand*: examples that are randomly selected; (b) *ques + img*: examples that are selected based on the joint similarity of question and image features, which is used in PICa; (c) *fused*: our default strategy that selects examples based on the similarity of fused features; (d) *fused + ques + img*: a combination of our default strategy and PICa’s strategy; and (e) *answer logits*: examples that are selected based on the similarity of answer logits obtained in (3). Besides the final accuracy, we also report the hit rate of answers within the selected examples for each strategy.

The results show that the accuracy is positively correlated with the hit rate of answers, which verifies our hypothesis that answer-aware examples contribute significantly to the performance of Prophet. Compared with other strategies, our default strategy (c) achieves the best performance with the highest hit rate. The strategy (d) that integrates other information (ques + img) into the (c) leads to worse performance due to the introduction of irrelevant and noisy information. Finally, strategy (e) reports slightly worse performance than (c). We conjecture that this is because the answer logits have lost too much information of the input question and image, which is also useful for GPT-3 to perform knowledge reasoning.

Numbers of examples and queries: Table I(e) contains the ablation studies for the numbers of examples and queries. We choose different numbers of examples $N \in \{0, 1, 8, 16, 20\}$ for each query and different numbers of queries $T \in \{1, 5\}$, respectively. The results show that the performance of Prophet improves with the increase of N and T , which is consistent with the results in PICa. By increasing T from 1 to 5, the entries

⁴In practice, multiple ground-truth answers are provided. If multiple answers are hit simultaneously, we choose the answer with the largest soft score for accumulation.

with larger N enjoy greater performance improvements at the expense of linearly increasing overheads.

Interestingly, the Prophet variant with $N=0$ delivers worse performance than the VQA model in stage-1 (49.97% vs. 53.04%), even though answer candidates are provided. Meanwhile, when given one example ($N=1$), the Prophet variant distinctly surpasses the VQA model (56.75% vs. 53.04%). This suggests the necessity of few-shot in-context examples for GPT-3 to activate its capability to adapt to the knowledge-based VQA task.

Prompt contents: In Table I(f), we ablate the prompt contents in the default settings by: (b) removing the prompt head; (c) removing the confidence scores for answer candidates; (d) removing image captions; and (e) adding predicted tags from external models [21].

The results lead to the following observations: First, the confidence scores are of critical importance to the performance of our Prophet. This is because they carry the necessary information for GPT-3 to understand the answer candidates. Second, without image captions, Prophet still works steadily. This reflects the fact that our answer heuristics in prompts already provide sufficient information for Prophet to solve the task. Third, the prompt head is of less importance, indicating that GPT-3 is capable of understanding the task directly from the in-context examples. Finally, introducing extra information like object tags leads to a slight performance drop, which is contrary to the results in PICA. We conjecture this information has already been encoded in answer heuristics implicitly.

Prediction behaviors in different stages: In Table I(b), we can observe a significant performance improvement of Prophet (stage-2) over its corresponding MCAN model (stage-1). To better understand this improvement, we conduct a statistical analysis of Prophet’s prediction behaviors. As Prophet takes K answer candidates from MCAN as inputs, we define three prediction behaviors of Prophet, namely “keep top 1”, “in top 2- K ”, and “beyond top K ”, which means LLM’s prediction hits the top-1 candidate, one of the top 2- K candidates, none of the K candidates, respectively. Note that all the testing samples can be categorized into one of the three classes above. The statistical results in Fig. 5 show that: (i) for 68.1% of the testing samples (green slice), Prophet keeps the top-1 predictions of MCAN. These samples achieve a 69% accuracy and are mostly easy samples; (ii) for 21.8% of the testing samples (blue slice), Prophet selects answers from the top 2- K answer candidates. These samples are relatively hard, so that MCAN delivers a 24% accuracy while Prophet has a much higher 40% accuracy; (iii) for the remaining 10.1% of the testing samples (yellow slice), Prophet resorts to the LLM to predict new answers beyond the answer candidates.⁵ For these challenging samples, MCAN only delivers a 12% accuracy while Prophet magnificently achieves a 42% accuracy. More analyses are provided in the appendix.

Different LLMs: In Table II, we investigate the effects of different LLMs by replacing the default GPT-3 (text-davinci-002) with the latest commercial and open-source models. From the

TABLE II
ABLATION STUDY OF DIFFERENT LLMs

LLM (version or size)	per-sample average cost	accuracy
<i>commercial models</i>		
GPT-3 (text-davinci-002)	\$0.2	60.8
GPT-3 (3.5-turbo-instruct)	\$0.015	58.9
<i>open-source models</i>		
LLaMA-2 (7B) [49]	2.7s	56.6
LLaMA-2-Chat (7B) [49]	2.7s	54.0
LLaMA-2 (13B) [49]	4.8s	57.9
LLaMA-2-Chat (13B) [49]	4.8	56.5
Mistral (7B) [50]	3.0s	59.7

All variants use the default settings and are evaluated on the testing set of OK-VQA. The per-sample average costs of the opensource models are measured by the GPU running time on a server with A100 GPUs while the costs of the commercial models are measured by money.

TABLE III
ABLATION STUDY FOR PROPHET++ ON THE TESTING SET OF OK-VQA

method	stage-1		stage-2		accu.
	VQA model	LMM	LLM/ LMM	pure text/ img+text	
GPT-4o	-	-	-	-	46.0
Prophet	MCAN	-	GPT-4o	pure-text	62.9
Prophet++	MCAN	GPT-4o	GPT-4o	pure-text	63.8
	MCAN	GPT-4o	GPT-4o	img+text	64.2

The first split contains the results of directprediction and Prophet-based prediction using the same LMM (GPT-4o). The second split contains two Prophet++ variants with different settings. $N=16$ is used for Prophet and Prophet++.

results, we have the following observations: (i) the capability of the default GPT-3 model consistently outperforms all the compared LLMs, including its accelerated variant (3.5-turbo-instruct); (ii) for the LLMs of the same version but different sizes (e.g., 7B and 13B LLaMA-2 models [49]), the large-size ones show better performance than the small-size ones at the expense of near-linearly increasing running time; (iii) the chat-oriented variants like LLaMA-2-Chat [49], which are additionally trained by instruction tuning and human feedback [51], deliver inferior performance to their non-chatty counterparts. This can be explained by the introduced *alignment tax* when aligning the model with human behaviors by RLHF; (iv) with only 7B model size, the latest LLM Mistral [50] reports near GPT-3 level performance, revealing the potential of open-source LLMs.

Effect of LMMs in Prophet++: In Table III, we investigate the effects of different strategies. The results suggest that: (i) although GPT-4o is a highly-capable LMM, directly using it in a direct-prediction manner achieves much worse performance than the Prophet-based counterpart (46.0 vs. 62.9), showing the effectiveness of Prophet’s two-stage prompting paradigm; (ii) by introducing the third type of answer heuristic in the first stage, Prophet++ delivers a 0.9-point improvement over Prophet, showing the effectiveness and complementarity of the answer-aware rationales; (iii) when visual signals are additionally utilized in the second stage, we obtain further a 0.4-point improvement, validating the effectiveness of multimodal prompting in Prophet++. More results with the latest LMMs are provided in the appendix.

⁵The probability that Prophet’s prediction is constituted of the combination of candidates is rare that can be neglected.

TABLE IV
COMPARISONS TO THE STATE-OF-THE-ART METHODS ON
OK-VQA TESTING SET

method	accuracy
<i>methods with external knowledge bases</i>	
KRISP [18]	38.9
Visual Retriever-Reader [54]	39.2
MAVE _x [19]	40.3
UnifER [55]	42.1
TRiG [20]	49.4
<i>methods with multimodal pretraining</i>	
Unified-IO (2.8B) [52]	54.0
Flamingo (80B) [11]	57.8
PALI (17B) [53]	64.5
<i>methods with GPT-3 API</i>	
PiCa [21]	48.0
KAT [†] [22]	53.1
REVIVE [†] [23]	56.6
PromptCap (OFA) [†] [38]	60.4
Prophet (MCAN)	61.1
Prophet (mPLUG)	62.5
<i>methods with GPT-4o API</i>	
Prophet++ (mPLUG)	65.7

The compared methods are split into three groups based on their knowledge resources and usages. [†]: method needs to query GPT-3 during training.

D. Main Results

For the comparisons below, we use all the default settings except the number of examples N . We set $N=20$ for OK-VQA and A-OKVQA and respectively set $N=7$ and $N=16$ for ScienceQA and TextVQA as they need extra hint and OCR tokens. By instantiating Prophet with two VQA models, we obtain Prophet (MCAN) and Prophet (mPLUG). Finally, we report the result of Prophet++ (mPLUG) on all the datasets to compare with Prophet (mPLUG).

Comparative results on OK-VQA: Table IV contains the comparisons of our Prophet and existing state-of-the-art methods on OK-VQA. The table is split into three sections. The first section lists the retrieval-based methods leveraging external KBs [18], [19], [20]. The second section contains the methods that are directly pretrained on a large-scale multimodal corpus [11], [52], [53]. The last two sections show the methods that incorporate the GPT-3 and GPT-4o [21], [22], [23], [38], which are accessible via online APIs.

Our Prophet belongs to the last section. It outperforms all the compared methods by a distinct margin. Prophet is 13.1 points higher than PiCa [21] when both methods use GPT-3 as the only knowledge resource. This confirms our hypothesis that the capacity of GPT-3 has not been fully activated in previous studies. Compared to KAT [22] and REVIVE [23], which utilize GPT-3 and other external KBs together in sophisticated systems, our Prophet is much simpler and more effective. Moreover, KAT, REVIVE, and PromptCap need to use GPT-3 to process all the training samples for their model training, which significantly increases the costs. In contrast, our Prophet only uses GPT-3 at inference time, which is more economical. Compared to the Flamingo-80B equipped with 32 in-context examples [11], Prophet (MCAN) delivers a significant performance improvement. Despite the fact that Prophet (MCAN) has

TABLE V
COMPARISONS TO THE STATE-OF-THE-ART METHODS ON A-OKVQA

method	DA		MC	
	val	test	val	test
ClipCap [16]	30.9	25.9	56.9	51.4
ViLBERT [16]	30.6	25.9	49.1	41.5
LXMERT [16]	30.7	25.9	51.4	41.6
KRISP [16]	33.7	27.1	51.9	42.2
GPV-2 [16]	48.6	40.7	60.3	53.7
Unified-IO [52]	-	45.2	-	-
MCAN [8]	52.0	45.6	-	-
mPLUG [40]	59.1	55.7	-	-
PromptCap (OFA) [38]	56.3	59.6	73.2	73.1
Prophet (MCAN)	58.2	55.7	76.4	73.6
Prophet (mPLUG)	64.7	58.5	76.6	75.1
Prophet++ (mPLUG)	68.3	68.0	87.7	86.7

DA and MC refer to the direct-answer and multiple-choice tasks, respectively. For the MC task, we devise a Prophet variant with a slightly different prompt.

a clear performance gap compared to PALI-17B [53], Prophet is more resource-efficient from the perspective of reproducibility.⁶ Moreover, by replacing MCAN with the pretrained generative model mPLUG, our method exhibits a 1.4-point further improvement, showing the substantial contribution of a powerful VQA model for Prophet. Finally, Prophet++ (mPLUG) equipped with a modern LMM delivers a 3.2-point improvement over Prophet (mPLUG).

Comparative results on A-OKVQA: Table V contains the comparative results on the challenging A-OKVQA dataset. The results on the DA task show that Prophet (MCAN) model significantly outperform the existing approaches including its base VQA model MCAN, showing its effectiveness and generalizability. Compared to the state-of-the-art method PromptCap [38] which also involves a pretrained VQA model OFA [12] and GPT-3, Prophet (MCAN) exhibits similar performance when using a weaker VQA model.

For the MC task, we introduce a Prophet variant with slightly modifying the prompt used in the original Prophet. In particular, we add the multiple-choice information into both the in-context examples and testing input to instruct GPT-3 to *choose* the correct one from four choices. Compared with all the methods, Prophet (MCAN) surpasses all the counterparts on the MC task, showing the flexibility and scalability of Prophet. Moreover, by introducing a more powerful VQA model mPLUG, Prophet (mPLUG) consistently outperforms Prophet (MCAN), while Prophet++ (mPLUG) further outperforms Prophet (mPLUG).

Results on ScienceQA and TextVQA: To verify the generalization ability of Prophet, we conduct experiments on two additional knowledge-based VQA datasets ScienceQA (IMG) and TextVQA, which require different types of knowledge (i.e., scientific knowledge and OCR knowledge) than that for OK-VQA and A-OKVQA. A Table VI shows that comparative results

⁶Flamingo-80B is trained on 1,536 TPUv4 for 15 days and PALI is trained on 1,024 TPUv4 for 7 days, which are unaffordable for most researchers. In contrast, Prophet (MCAN) uses one RTX-3090 to train a VQA model for 4 days and a certain number of GPT-3 invocations.

TABLE VI
COMPARISONS TO THE STATE-OF-THE-ART METHODS ON SCIENCEQA AND TEXTVQA

method	accu.	method	accu.
MCAN [8]	51.2	LoRRA [26]	27.6
mPLUG [40]	77.0	M4C [59]	40.5
InstructBLIP [56]	79.5	PromptCap [38]	51.9
LLaMA-Adapter [57]	80.3	mPLUG [40]	53.5
MM-CoT [58]	82.9	TAP [43]	54.0
Human Average [25]	87.5	Flamingo-80B [11]	54.1
LLaVa [42]	88.0	LaTr [44]	59.6
Prophet (mPLUG)	88.2	Prophet (mPLUG)	61.3
Prophet++ (mPLUG)	90.5	Prophet++ (mPLUG)	61.8

(a) ScienceQA (IMG)

(b) TextVQA

of Prophet and existing state-of-the-art methods on respective datasets. As we have witnessed the steady improvements of mPLUG over MCAN, we only report the results for Prophet (mPLUG) on these two datasets. Specifically, Prophet surpasses all the counterparts on ScienceQA (IMG), including the base VQA model mPLUG, the average human performance [25], and the latest LMM LLaVa trained with visual instruction tuning [42]. On TextVQA, Prophet outperforms the published state-of-the-art methods, including the methods with text-aware or layout-aware pretraining on large-scale scene-text image datasets [43], [44]. Finally, the Prophet++ models prominently outperform their corresponding Prophet counterparts on respective datasets.

V. CONCLUSION

In this paper, we present Prophet—a conceptually simple framework which uses LLMs as the knowledge engine for knowledge-based VQA. To better activate the few-shot learning capacity of LLMs, we introduce a novel paradigm to prompt LLMs with complementary answer heuristics. Extensive ablations, comparative experiments, and comprehensive analyses on four diverse knowledge-based VQA datasets show the superiority of Prophet over all existing state-of-the-art methods. Notably, Prophet can be integrated with varied combinations of VQA models, LLMs, and even LMMs, showing the flexibility, scalability, and generalizability of our framework.

REFERENCES

- [1] T. Brown et al., “Language models are few-shot learners,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [2] P. Anderson et al., “Bottom-up and top-down attention for image captioning and visual question answering,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6077–6086.
- [3] J. Yu, J. Li, Z. Yu, and Q. Huang, “Multimodal transformer with multi-view visual representation for image captioning,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 12, pp. 4467–4480, Dec. 2020.
- [4] R. Hong, D. Liu, X. Mo, X. He, and H. Zhang, “Learning to compose and reason with language tree structures for visual grounding,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 2, pp. 684–696, Feb. 2022.
- [5] C. Deng, Q. Wu, F. Hu, F. Lyu, and M. Tan, “Visual grounding via accumulated attention,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1670–1684, Mar. 2022.
- [6] J. Deng et al., “TransVG++: End-to-end visual grounding with language conditioned vision transformer,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13636–13652, Nov. 2023.
- [7] J.-H. Kim, J. Jun, and B.-T. Zhang, “Bilinear attention networks,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 1571–1581.
- [8] Z. Yu, J. Yu, Y. Cui, D. Tao, and Q. Tian, “Deep modular co-attention networks for visual question answering,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6281–6290.
- [9] H. Tan and M. Bansal, “LXMERT: Learning cross-modality encoder representations from transformers,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2019, pp. 5100–5111.
- [10] X. Li et al., “OSCAR: Object-semantics aligned pre-training for vision-language tasks,” in *Proc. Comput. Vis. 2020: 16th Eur. Conf.*, Glasgow, UK, Springer, Aug. 23–28, 2020, pp. 121–137.
- [11] J.-B. Alayrac et al., “Flamingo: A visual language model for few-shot learning,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 23716–23736.
- [12] P. Wang et al., “OFA: Unifying architectures, tasks, and modalities through a simple sequence-to-sequence learning framework,” in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 21218–21340.
- [13] P. Wang, Q. Wu, C. Shen, A. Dick, and A. Van Den Hengel, “FVQA: Fact-based visual question answering,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 10, pp. 2413–2427, Oct. 2018.
- [14] P. Wang, Q. Wu, C. Shen, A. R. Dick, and A. van den Hengel, “Explicit knowledge-based reasoning for visual question answering,” in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 1290–1296.
- [15] K. Marino, M. Rastegari, A. Farhadi, and R. Mottaghi, “Ok-VQA: A visual question answering benchmark requiring external knowledge,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3195–3204.
- [16] D. Schwenk, A. Khandelwal, C. Clark, K. Marino, and R. Mottaghi, “A-OKVQA: A benchmark for visual question answering using world knowledge,” in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2022, pp. 146–162.
- [17] H. Liu and P. Singh, “ConceptNet: A practical commonsense reasoning tool-kit,” *BT Technol. J.*, vol. 22, no. 4, pp. 211–226, 2004.
- [18] K. Marino, X. Chen, D. Parikh, A. Gupta, and M. Rohrbach, “KRISP: Integrating implicit and symbolic knowledge for open-domain knowledge-based VQA,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 14111–14121.
- [19] J. Wu, J. Lu, A. Sabharwal, and R. Mottaghi, “Multi-modal answer validation for knowledge-based VQA,” in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2022, pp. 2712–2721.
- [20] F. Gao, Q. Ping, G. Thattai, A. Reganti, Y. N. Wu, and P. Nataraajan, “Transform-retrieve-generate: Natural language-centric outside-knowledge visual question answering,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5067–5077.
- [21] Z. Yang et al., “An empirical study of GPT-3 for few-shot knowledge-based VQA,” in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2022, pp. 3081–3089.
- [22] L. Gui, B. Wang, Q. Huang, A. Hauptmann, Y. Bisk, and J. Gao, “KAT: A knowledge augmented transformer for vision-and-language,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2021, pp. 956–968.
- [23] Y. Lin, Y. Xie, D. Chen, Y. Xu, C. Zhu, and L. Yuan, “REVIVE: Regional visual representation matters in knowledge-based visual question answering,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 10560–10571.
- [24] Z. Shao, Z. Yu, M. Wang, and J. Yu, “Prompting large language models with answer heuristics for knowledge-based visual question answering,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14974–14983.
- [25] P. Lu et al., “Learn to explain: Multimodal reasoning via thought chains for science question answering,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 2507–2521.
- [26] A. Singh et al., “Towards VQA models that can read,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 8317–8326.
- [27] P. Zhang et al., “VinVL: Revisiting visual representations in vision-language models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5579–5588.
- [28] S. Shen et al., “How much can clip benefit vision-and-language tasks?,” in *Proc. Int. Conf. Learn. Representations*, 2022.
- [29] L. Li, Z. Gan, Y. Cheng, and J. Liu, “Relation-aware graph attention network for visual question answering,” in *Proc. Int. Conf. Comput. Vis.*, 2019, pp. 10313–10322.
- [30] J. Li, D. Li, C. Xiong, and S. Hoi, “BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 12888–12900.
- [31] J. Lu, D. Batra, D. Parikh, and S. Lee, “ViLBERT: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks,” in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 13–23.
- [32] Y.-C. Chen et al., “Uniter: Universal image-text representation learning,” in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 104–120.

- [33] Y. Cui et al., "Rosita: Enhancing vision-and-language semantic alignments via cross-and intra-modal knowledge integration," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 797–806.
- [34] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [35] J. Johnson, B. Hariharan, L. V. D. Maaten, L. Fei-Fei, C. L. Zitnick, and R. Girshick, "CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2901–2910.
- [36] D. A. Hudson and C. D. Manning, "Gqa: A new dataset for real-world visual reasoning and compositional question answering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6700–6709.
- [37] D. Vrandečić and M. Krötzsch, "Wikidata: A free collaborative knowledgebase," *Commun. ACM*, vol. 57, no. 10, pp. 78–85, 2014.
- [38] Y. Hu, H. Hua, Z. Yang, W. Shi, N. A. Smith, and J. Luo, "PromptCap: Prompt-guided image captioning for VQA with GPT-3," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 2963–2975.
- [39] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 4171–4186.
- [40] C. Li et al., "mplug: Effective and efficient vision-language learning by cross-modal skip-connections," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2022, pp. 7241–7259.
- [41] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 3104–3112.
- [42] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," 2023, *arXiv:2304.08485*.
- [43] Z. Yang et al., "Tap: Text-aware pre-training for text-VQA and text-caption," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8751–8761.
- [44] A. F. Biten, R. Litman, Y. Xie, S. Appalaraju, and R. Manmatha, "LATR: Layout-aware transformer for scene-text VQA," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16548–16558.
- [45] A. F. Biten et al., "Scene text visual question answering," in *Proc. Int. Conf. Comput. Vis.*, 2019, pp. 4291–4301.
- [46] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [47] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, "Making the V in VQA matter: Elevating the role of image understanding in visual question answering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6325–6334.
- [48] R. Krishna et al., "Visual genome: Connecting language and vision using crowdsourced dense image annotations," *Int. J. Comput. Vis.*, vol. 123, no. 1, pp. 32–73, 2017.
- [49] H. Touvron et al., "LLAMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [50] M. AI, "Mistral 7B," 2023, *arXiv:2310.06825*.
- [51] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.
- [52] J. Lu, C. Clark, R. Zellers, R. Mottaghi, and A. Kembhavi, "Unified-IO: A unified model for vision, language, and multi-modal tasks," 2022, *arXiv:2206.08916*.
- [53] X. Chen et al., "PALI: A jointly-scaled multilingual language-image model," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [54] M. Luo, Y. Zeng, P. Banerjee, and C. Baral, "Weakly-supervised visual-retriever-reader for knowledge-based question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 6417–6431.
- [55] Y. Guo, L. Nie, Y. Wong, Y. Liu, Z. Cheng, and M. Kankanhalli, "A unified end-to-end retriever-reader framework for knowledge-based VQA," in *Proc. ACM Multimedia*, 2022, pp. 2061–2069.
- [56] W. Dai et al., "InstructBLIP: Towards general-purpose vision-language models with instruction tuning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 49250–49267.
- [57] R. Zhang et al., "Llama-adaptor: Efficient fine-tuning of language models with zero-init attention," 2023, *arXiv:2303.16199*.
- [58] Z. Zhang, A. Zhang, M. Li, H. Zhao, G. Karypis, and A. Smola, "Multimodal chain-of-thought reasoning in language models," 2023, *arXiv:2302.00923*.
- [59] R. Hu, A. Singh, T. Darrell, and M. Rohrbach, "Iterative answer prediction with pointer-augmented multimodal transformers for textVQA," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9989–9999.



Zhou Yu (Member, IEEE) received the BEng and PhD degrees from Zhejiang University, China, in 2010 and 2015, respectively. He is currently a full professor with the School of Computer Science and Technology, Hangzhou Dianzi University, China. His research interests include multimodal analysis, visual-language learning, and large multimodal models. He has authored and co-authored more than 40 scientific articles and has also served as an invited reviewer or a program committee members for top conferences and prestigious journals including *AAAI*, *IJCAI*, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Circuits and Systems for Video Technology*, and *IEEE Transactions on Neural Networks and Learning Systems*.



Xuecheng Ouyang received the BEng degree from the School of Computer and Information Engineering, Tianjin Normal University, Tianjin, China, in 2022. He is currently working toward the MEng degree with the HDU-ITMO Joint Institute, Hangzhou Dianzi University, Hangzhou, China. His current research interests include vision-language learning, multimodal reasoning and large multimodal models.



Zhenwei Shao received the BEng degree from the School of Computer Science and Technology, Hangzhou Dianzi University, Hangzhou, China, in 2021. He is currently working toward the PhD degree with the School of Computer Science and Technology, Hangzhou Dianzi University. His current research interests include multimodal analysis, deep learning, and large multimodal models.



Meng Wang (Fellow, IEEE) received the BE and PhD degrees from the Special Class for the Gifted Young and the Department of Electronic Engineering and Information Science, University of Science and Technology of China (USTC), Hefei, China, respectively. He is a professor with the Hefei University of Technology, China. His current research interests include multimedia content analysis, search, mining, recommendation, and large-scale computing. He received the best paper awards successively from the 17th and 18th ACM International Conference on Multimedia, the best paper award from the 16th International Multimedia Modeling Conference, the best paper award from the 4th International Conference on Internet Multimedia Computing and Service, and the best demo award from the 20th ACM International Conference on Multimedia.



Jun Yu (Senior Member, IEEE) received the BEng and PhD degrees from Zhejiang University, Zhejiang, China. He was an associate professor with the School of Information Science and Technology, Xiamen University, Xiamen, China. From 2009 to 2011, he was with Nanyang Technological University, Singapore. From 2012 to 2013, he was a visiting researcher with Microsoft Research Asia (MSRA). He is currently a professor with the department of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen, China. He has authored or co-authored more than 100 scientific articles. Over the past years, his research interests have included multimedia analysis, machine learning, and image processing. In 2017, he received the IEEE SPS Best Paper Award. He has (co-)chaired several special sessions, invited sessions, and workshops. He has served as a program committee member for top conferences including *CVPR*, *ACM Multimedia*, *AAAI*, *IJCAI*, and has served as associate editors for prestigious journals including *IEEE Transactions on Circuits and Systems for Video Technology* and *Pattern Recognition*.